Рубежный контроль № 1

Дисциплина: «Парадигмы и конструкции языков программирования»

Вариант запросов: Г

Предметная область № 1: «Студент — Группа»

Автор: Лимаренко Дарья, группа ИУ5-34

Исходный код main.py

```
from operator import itemgetter


class Student:
    """Студент учебной группы."""

    def __init__(self, id, surname, scholarship, group_id):
        self.id = id
        self.surname = surname
        self.scholarship = scholarship
        self.group_id = group_id


class Group:
    """Учебная группа."""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class StudentGroup:
    """Связь студентов и групп для отношения многие-ко-многим."""

    def __init__(self, student_id, group_id):
        self.student_id = student_id
        self.group_id = group_id


groups = [
    Group(1, "Аналитика-01"),
    Group(2, "Архитектура-02"),
    Group(3, "ИУ5-34"),
    Group(4, "Программирование-04"),
]

students = [
    Student(1, "Иванов", 2500, 1),
    Student(2, "Петров", 3200, 1),
    Student(3, "Сидорова", 1800, 2),
    Student(4, "Кузнецов", 3200, 3),
    Student(5, "Смирнова", 2200, 3),
]

student_groups = [
    StudentGroup(1, 1),
    StudentGroup(1, 3),
    StudentGroup(2, 1),
    StudentGroup(2, 2),
    StudentGroup(3, 2),
    StudentGroup(3, 4),
    StudentGroup(4, 3),
    StudentGroup(5, 3),
    StudentGroup(5, 4),
]


def get_query_results():
    """Выполнение запросов варианта Г."""

    # Соединение данных один-ко-многим по полю group_id.
    one_to_many = [
        (group.name, student.surname, student.scholarship)
        for group in groups
```

```
        for student in students
        if student.group_id == group.id
    ]

    # Соединение данных многие-ко-многим через класс StudentGroup.
    many_to_many_temp = [
        (group.name, link.student_id)
        for group in groups
        for link in student_groups
        if group.id == link.group_id
    ]
    many_to_many = [
        (student.surname, group_name)
        for group_name, student_id in many_to_many_temp
        for student in students
        if student.id == student_id
    ]

    groups_starting_with_a = list(filter(lambda group: group.name.startswith("A"), groups))
    res_1 = {
        group.name: [student.surname for student in students if student.group_id == group.id]
        for group in groups_starting_with_a
    }

    max_scholarship_by_group = []
    for group in groups:
        group_students = list(filter(lambda row: row[0] == group.name, one_to_many))
        if len(group_students) > 0:
            scholarships = [scholarship for _, _, scholarship in group_students]
            max_scholarship = max(scholarships)
            max_scholarship_by_group.append((group.name, max_scholarship))
    res_2 = sorted(max_scholarship_by_group, key=itemgetter(1), reverse=True)

    res_3 = sorted(many_to_many, key=itemgetter(1))

    return res_1, res_2, res_3


def main():
    """Вывод результатов запросов варианта Г."""

    res_1, res_2, res_3 = get_query_results()
    print("Задание Г1")
    print(res_1)
    print("\nЗадание Г2")
    print(res_2)
    print("\nЗадание Г3")
    print(res_3)


if __name__ == "__main__":
    main()
```

Фактический вывод программы

```
Задание Г1
{'Аналитика-01': ['Иванов', 'Петров'], 'Архитектура-02': ['Сидорова']}

Задание Г2
[('Аналитика-01', 3200), ('ИУ5-34', 3200), ('Архитектура-02', 1800)]

Задание Г3
[('Иванов', 'Аналитика-01'), ('Петров', 'Аналитика-01'), ('Петров', 'Архитектура-02'), ('Сидорова',
'Архитектура-02'), ('Иванов', 'ИУ5-34'), ('Кузнецов', 'ИУ5-34'), ('Смирнова', 'ИУ5-34'), ('Сидорова',
'Программирование-04'), ('Смирнова', 'Программирование-04')]
```